Author: Brendan Bianchi

Cache & Memory Performance Profiling – Project Report

AI Usage Statement

Claude was used for code creation, plot generation, assistance in data analysis, and writing support.

Abstract

This project characterizes the memory hierarchy of an AMD Ryzen 5 3600 (Zen 2) running Windows 11, measuring the latency, bandwidth, concurrency and miss behavior of each level from L1 to DRAM. I built a reproducible suite of C++ microbenchmarks: a dependent pointer chase for latency, multithreaded bandwidth kernels with pattern, granularity and read/write controls, a loaded-latency test, and two arithmetic kernels whose locality I varied at constant work. AMD uProf supplied hardware counters. Load-to-use latency was 4.0 cycles in L1, 11.8 in L2, about 40–47 in L3, and 91 ns in DRAM with 2 MB pages (110 ns with 4 KB pages). All six cores sustained about 34–36 GB/s, 80–84% of the 42.7 GB/s theoretical peak, while a single core kept only about 17 misses in flight, which by Little's Law limits it to roughly 12 GB/s for random access. Page walks with 4 KB pages cost about 20 ns per access at a 1 GiB footprint. Overall, single-thread memory performance is bounded by latency and per-core concurrency, and multi-thread performance by DRAM bandwidth and queueing.

1. Introduction and goals

Modern processors can execute several instructions per cycle, but a single load that misses all caches takes hundreds of cycles to return. As a result, most real programs are limited by how fast data moves through the memory hierarchy rather than by arithmetic. Each level of that hierarchy trades capacity for speed: L1 is tiny and answers in a few cycles, DRAM holds gigabytes but takes around 100 ns. Writing fast software therefore requires knowing these numbers for the actual machine, not from a datasheet.

This project measures them on my own desktop. It asks four questions: what the zero-queue latency of each level is; how access pattern, access size and the read/write mix change achievable bandwidth; where latency starts to rise with load and whether Little's Law explains that point; and how cache and TLB misses translate into the runtime of simple kernels. Along the way, the goal is to practice measurement discipline: controlling the environment, validating that each benchmark measures what it claims, reporting uncertainty, and explaining results that don't match expectations.

The report first states my hypotheses, then describes the setup and validation. Sections 5–11 present one experiment each, Section 12 covers benchmark pitfalls and a failure case, Section 13 lists anomalies and limitations, and Section 14 concludes.

Required experiments and where each is answered:

Assignment requirement	Section	Figure(s)
Zero-queue latency per level	5	fig1
Pattern and granularity sweep	6	fig2, fig3

Author: Brendan Bianchi

Assignment requirement	Section	Figure(s)
Read/write mix (100R, 100W, 70/30, 50/50)	7	fig4
Intensity sweep, knee, Little's Law	8	fig5, fig6
Working-set sweep (L1/L2/LLC/DRAM regions)	9	fig1, fig7
Cache-miss impact with counters	10	fig8
TLB-miss impact with huge pages	11	fig9
Benchmark pitfall / failure case	12	fig10

2. Hypotheses (written before the experiments)

These predictions were written before any experiment was run, so the results could be judged against them rather than explained after the fact. The table summarizes each prediction and its outcome; the reasoning behind each follows.

ID	Prediction	Measured	Verdict
H1	L1 4–5 cycles, L2 12–16 cycles, LLC 40–70 cycles, DRAM 70–110 ns	4.0 / 11.8 / 40–47 cycles / 91–110 ns	Supported
H2	4 KB pages add latency above a few MiB; 2 MB pages remove it; no effect inside TLB reach	+19 ns at 1 GiB; 0 ns at 16 pages	Supported
H3	Sequential 64 B chase 10–30× faster than random at DRAM size; benefit fades by 1024 B	4.4 ns vs 110 ns (25×); 9.4 ns at 1024 B	Supported
H4	Useful bandwidth of sparse access falls with bytes used per line; random large blocks approach sequential	8 B stride: 1.9 vs 23 GB/s; rand 4 KB: 12 GB/s (1 thread)	Partly (see 128 B anomaly)
H5	Read > 70/30 ≈ 50/50 > write; NT stores recover writes	34.6 > 22.5 > 18.7 > 13.5 GB/s; NT write 21.1	Supported
H6	Per-core MLP limit about 10–16 misses; system knee at 60–80% of peak	16.9 lines per core; read knee 34.8 GB/s = 82% of 42.7 GB/s peak	Mostly supported (both slightly above range)
H7	One bandwidth plateau per level; DRAM 70–85% of theoretical peak	1 thread: 109 / 78 / 23 GB/s (L2 / L3 / DRAM); all cores 34.3	Mostly supported (L1 and L2

Author: Brendan Bianchi

ID	Prediction	Measured	Verdict
		GB/s = 80% of 42.7 GB/s peak	merge)
H8	Stride 8 and 512 raise misses and runtime about 10×; random slowest	mul: 1.27 → 10.5 → 18.5 ns/elem; rand 26.3	Supported

H1 – Per-level latency. I expected the random-chase latency to form plateaus of about 4–5 cycles for L1, 12–16 cycles for L2, 40–70 cycles for L3 and 70–110 ns for DRAM. L1 and L2 latencies are fixed by pipeline depth; the L3 adds a trip across the core complex to the right slice; DRAM adds the Infinity Fabric, the memory controller and the DRAM row access itself. The steps should sit near each cache's capacity but be smeared, because some accesses still hit a smaller level.

H2 – Page size. Above a few MiB, 4 KB pages should add latency because the TLB can no longer cover the footprint and every access needs a page walk. With 2 MB pages one TLB entry covers 512 times more memory, so most of that cost should disappear. Inside the TLB's reach the page size should make no difference.

H3 – Pattern. A sequential chase should look 10–30× faster than a random one at DRAM sizes, because the hardware prefetchers recognize the stream and fetch ahead. The advantage should fade at large strides, since prefetchers have limited lookahead and stop at page boundaries.

H4 – Granularity. When a program uses only 8 bytes of every 64-byte line, its useful bandwidth should fall to about 1/8 of the line bandwidth, while the hardware still moves whole lines. Random accesses of 1 KB or more should approach sequential bandwidth because each DRAM access is amortized over many bytes.

H5 – Read/write mix. Pure reads should be fastest and pure writes slowest, with 70/30 and 50/50 in between. A normal store first reads the line into the cache (read-for-ownership) and later writes it back, so each written byte costs two transfers; the DRAM bus also loses time switching between reads and writes. Non-temporal stores, which skip the read, should recover most of the write deficit.

H6 – Intensity. A single core should scale almost linearly with the number of independent misses up to its miss-buffer limit (I guessed 10–16), then flatten while latency grows. System-wide, latency should stay near idle until about 60–80% of peak bandwidth, then rise steeply because requests queue at the memory controller.

H7 – Working set. Sequential bandwidth should show one plateau per level, dropping roughly 2× at each boundary, and all cores together should reach 70–85% of the theoretical DRAM peak.

H8 – Kernel locality. With the same work, strides that touch a new line (stride 8) or a new page (stride 512) on every access should raise misses per element and runtime by about an order of magnitude, with random order the slowest.

The predicted ranges reflect typical published figures for this class of processor and memory. The results that departed most from them were the knee positions for write-heavy mixes and the small benefit of huge pages for streaming kernels; both are discussed in the results.

Author: Brendan Bianchi

3. Experimental setup and methodology

3.1 System configuration

Item	Value
CPU	AMD Ryzen 5 3600 (Zen 2, family 17h model 71h), 6 cores / 12 threads, 2 CCX
Caches	L1d 32 KiB and L2 512 KiB per core; L3 16 MiB per CCX (32 MiB total)
Memory	2 × 16 GiB DDR4 (G.Skill F4-3200C16), dual channel, running at 2667 MT/s (rated 3200; XMP not enabled) → theoretical peak 2 × 2667 × 8 B = 42.7 GB/s
OS	Windows 11 Pro, build 26200
Toolchain	MSYS2 UCRT64, g++ 16.2.0, `-O2 -march=native -std=c++17`, static link
Analysis	Python 3.14 with numpy, pandas, matplotlib
Power	Balanced plan, processor boost off, min/max processor state 100% (AC)
Calibrated core clock	3.552 GHz measured (nominal 3.593 GHz), CV 0.1% across latency runs
SMT	On
Pinning	Measurement thread on CPU 2; bandwidth threads one per physical core (6)
Large pages	"Lock pages in memory" granted; coverage verified per buffer (`huge_kb`)
Counters	AMD uProf 5.3.521, AMDuProfCLI `data_access` configuration (event-based sampling)
Memory integrity (VBS)	Off during measurement (required for uProf sampling)
Repetitions	5 repetitions × 200 ms per configuration

3.2 Environment control

Because memory latency is reported both in nanoseconds and in core cycles, the core clock had to be constant for the whole run. I disabled processor boost and set the minimum and maximum processor state to 100% on AC power, which kept the calibrated clock at 3.552 GHz with only 0.1% variation across all latency runs. Every benchmark thread was pinned to a specific logical CPU with `SetThreadGroupAffinity`, so the scheduler could not migrate it mid-measurement and invalidate its

cache contents. Bandwidth tests used one thread per physical core, so that two threads never shared a core through SMT. I closed other programs, ran on AC power and paused Windows Update during the run. A setup script saved the original power settings and a restore script put them back afterwards.

3.3 Measurement techniques

Pointer chase. The latency benchmark links the cache lines of a buffer into one randomly permuted cycle and follows the pointers. Each load needs the address returned by the previous one, so loads cannot overlap and the prefetchers cannot guess the next address; the time per hop is the true load-to-use latency. Variants walk the lines sequentially at a fixed stride (to show what prefetching hides) or place one line per 4 KB page (to isolate TLB behavior).

Multiple chains. To measure memory-level parallelism, the same benchmark interleaves K independent chains. The core can then have up to K misses in flight, and throughput versus K shows how many outstanding misses it supports.

Bandwidth. Each thread gets its own buffer, which it initializes itself so the pages are allocated near the core that uses them. Knobs select sequential, strided or random order, the access size, and the read/write mix, applied in repeating groups of 10 accesses; an option uses non-temporal stores.

Loaded latency. One thread runs the random chase on an idle core while the other five cores generate memory traffic. A delay loop between generator accesses is swept from long to zero, raising the offered load step by step while the chase measures the latency it sees.

Kernels. `z = x*y` and saxpy (`y = a*x + y`) process the same elements with the same arithmetic, visited in unit-stride, strided (2–512) or random order, so any change in runtime comes from locality alone.

Clock calibration. Each run times a dependent chain of register-to-register additions, which takes exactly one cycle each, to measure the actual core frequency; nanoseconds are converted to cycles with that value rather than the nominal clock.

3.4 Statistics and uncertainty

Every configuration was repeated five times for 200 ms each. Tables report the mean with a 95% confidence interval from Student's t-distribution, which is appropriate for so few samples. A single run can be distorted by an interrupt or a background task; repetitions expose that as a wide interval, and several of the anomalies in Section 13 were first noticed this way.

3.5 Hardware counters

Hardware counters were collected with AMD uProf's command-line profiler using its `data_access` configuration. uProf samples: it records one event every N occurrences, so counts are reconstructed by multiplying samples by the interval. Because the profiled run also includes allocation, initialization and validation, each configuration was profiled twice, once normally and once with zero timed repetitions, and the difference was divided among the repetitions. The configuration monitors ten events on six hardware counters, so the hardware time-shares them; I corrected each count by the fraction of run time its event was actually monitored. The events used were L1 data-cache misses, L2 misses that go to the L3, demand fills from DRAM (the L3 misses), and L2 DTLB misses, which correspond to page walks.

Sampling adds noise, so small differences between configurations are only meaningful when they exceed the runtime confidence intervals.

4. Correctness validation

All 16 automated sanity checks passed in the final run.

Check	Result
Pointer chains form one cycle over all nodes	Pass
Core clock stable across latency runs	Pass (3.552 GHz, CV 0.1%)
Large pages actually granted	Pass (100% coverage of ≥ 8 MiB buffers)
Latency ordering L1 < L2 < L3 < DRAM (4 KB)	Pass (1.1 < 3.3 < 13.2 < 110.0 ns)
Latency ordering L1 < L2 < L3 < DRAM (2 MB)	Pass (1.1 < 3.3 < 11.2 < 90.8 ns)
L1 load-to-use 3–7 cycles	Pass (4.01 and 4.02 cycles)
DRAM point well beyond LLC	Pass (1 GiB vs 16 MiB)
Bandwidth buffers on large pages	Pass (100% coverage)
Single-thread R/W mixes memory-bound	Pass (all 4 mixes below 80% of the L2-resident ceiling)
Kernel results match a reference loop	Pass (kernels and kernels_tlb)
Every configuration ran	Pass (no entries in `failures.txt`)

Counter validation (uProf probe, `mul` kernel, 256 MiB):

Stride	Expected cycles/elem	Counted cycles/elem	dTLB misses/elem
1	4.64	4.86	0.003
512	63.8	67.6	1.16

Excerpt from `data/raw/validation.txt` (every line of the file reports OK):

```
  bw selftest seq read sum == n(n-1)/2 (fast path)              OK
  bw selftest stride-256 read sum matches closed form           OK
  bw selftest non-temporal write stores expected values         OK
  bw selftest mix 70/30 generic path writes 300 of 1000 blocks  OK
  bw selftest random offsets in bounds and aligned              OK
bandwidth selftest: OK
latency check pattern=rand pages=4k size=8388608 nodes=131072: OK
latency check pattern=page pages=4k size=8388608 nodes=2048: OK
latency check pattern=rand pages=large size=8388608 nodes=131072: OK
kernel check mul order=stride stride=512 pages=4k n=262144: OK
kernel check saxpy order=rand stride=1 pages=large n=393216: OK
```

Author: Brendan Bianchi

A memory benchmark can produce plausible-looking numbers while measuring the wrong thing, so the suite checks itself before any result is trusted. Each pointer chain is walked once to confirm it is a single cycle through every node; a chain that split into small loops would stay in cache and report a falsely low latency. The calibrated clock was stable to 0.1%, so the cycle conversions are consistent across the run. The large-page buffers were verified with `QueryWorkingSetEx`, which showed 100% of every buffer of 8 MiB or more actually backed by 2 MB pages. The kernels compare every output element against a plain sequential reference loop, and a self-test script exercises each benchmark on known inputs before the experiments start.

The results also agree with external references. The measured L1 latency of 4.0 cycles matches AMD's documented 4–5 cycles, and the L2 and L3 values fall within the ranges reported for Zen 2. The counter pipeline was checked the same way: on the `mul` kernel, the counted cycles per element (4.86 and 67.6) are within about 6% of runtime multiplied by clock frequency (4.64 and 63.8). That shows the sample scaling and the differential method recover real event counts. In the final run all 16 sanity checks passed and no configuration failed.

5. Zero-queue latency per level

Random pointer chase on an idle system, working set at or below half of each level's capacity:

Level	Working set	4 KB pages (ns)	4 KB (cycles)	2 MB pages (ns)	2 MB (cycles)	AMAT-corrected, 2 MB (cycles)
L1	16 KiB	1.13 ± 0.00	4.01	1.13 ± 0.00	4.02	4.02
L2	256 KiB	3.31 ± 0.01	11.76	3.31 ± 0.02	11.76	12.87
L3	8 MiB	13.18 ± 1.49	46.80	11.19 ± 0.79	39.79	41.62
DRAM	1 GiB	109.98 ± 1.07	390.84	90.85 ± 0.82	322.91	327.40

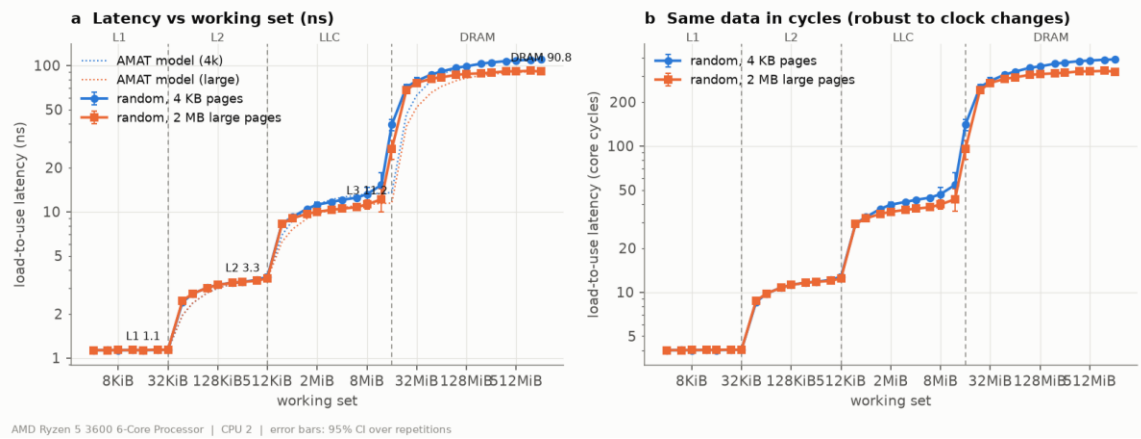


Figure 1: Random-chase latency vs working-set size with 4 KB and 2 MB pages; dashed lines mark the L1, L2 and L3 capacities.

Author: Brendan Bianchi

The random pointer chase measures true load-to-use latency because each load depends on the previous one: the core cannot overlap accesses, and the random order defeats the prefetchers. To read off each level, I used a working set no larger than half of that level's capacity, which keeps conflict misses rare. Even so, a random working set larger than one cache still finds some of its lines in the smaller caches above it. The AMAT correction removes that share using a uniform-random model, which is why the corrected L2 and L3 values are slightly higher than the raw ones.

Each level is markedly slower than the one before: about 3× from L1 to L2, 3.5× from L2 to L3, and 8× from L3 to DRAM. The L2 adds a larger, slower array behind the L1; the L3 is split into slices shared by the three cores of a core complex (CCX); and a DRAM access crosses the Infinity Fabric to the memory controller and waits for the DRAM row to open. At 322–391 cycles, a single DRAM miss costs as much time as hundreds of instructions.

Page size matters even for the "pure" latency numbers. At 1 GiB, 4 KB pages make each access 19.1 ns (21%) slower than 2 MB pages, because the TLB can no longer cover the footprint and the page walk is added to every load. A benchmark that ignores page size would report that walk as part of memory latency. The same effect is visible on a smaller scale at 8 MiB in L3 (13.2 vs 11.2 ns).

The AMAT model predicts the whole latency curve within 6.4% (4 KB) and 8.3% (2 MB) on average; it deviates most near the capacity boundaries, where real replacement is not uniform. These results support H1: every level fell within or close to the predicted range.

6. Access pattern and granularity

6.1 Latency by pattern (1 GiB working set)

Pattern	Latency (ns)
Sequential chase, 64 B stride	4.4
Sequential chase, 256 B stride	4.8
Sequential chase, 1024 B stride	9.4
Random chase	110.0

Author: Brendan Bianchi

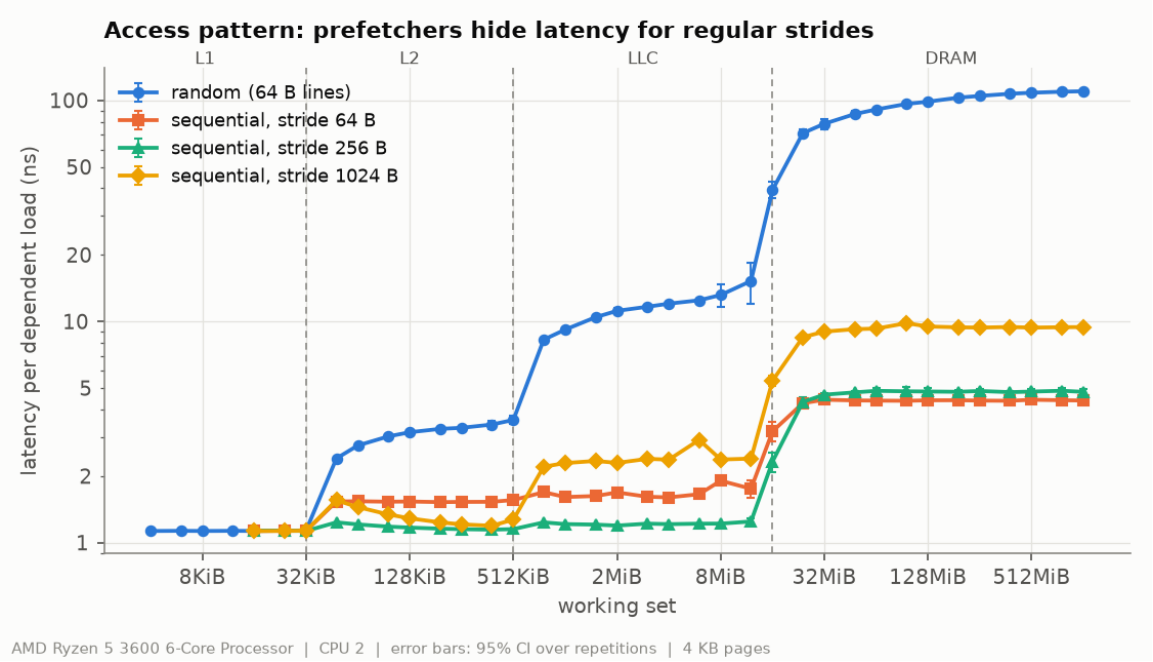


Figure 2: Chase latency by access pattern and stride (4 KB pages).

6.2 Bandwidth by pattern and granularity (DRAM-sized buffers)

Threads	Pattern	Access size	Useful GB/s	Line GB/s
1	Sequential	64 B	23.0 ± 1.4	23.0
1	Stride 64 B	8 B	1.86	14.9
1	Stride 4096 B	8 B	0.50	4.0
1	Random	8 B	0.27	2.1
1	Random	64 B	4.40	4.4
1	Random	128 B	3.22	3.2
1	Random	4096 B	12.0	12.0
6	Sequential	64 B	34.1 ± 2.1	34.1
6	Stride 64 B	8 B	4.33	34.7
6	Stride 4096 B	8 B	2.41	19.3
6	Random	8 B	1.23	9.8
6	Random	64 B	16.3	16.3
6	Random	4096 B	25.5	25.5

Full table: `data/processed/bw_pattern_summary.csv`.

Author: Brendan Bianchi

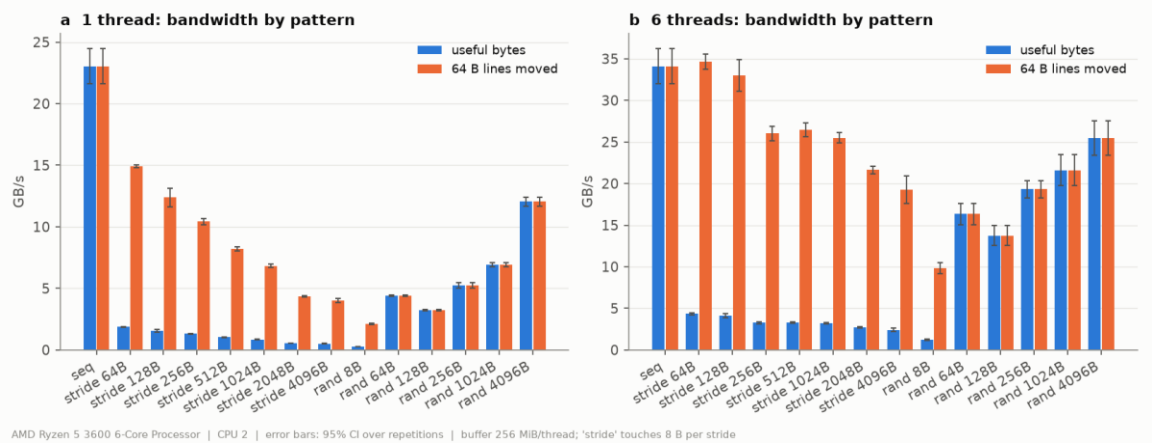


Figure 3: Useful and line bandwidth vs stride and random access size, 1 and 6 threads.

At the same 1 GiB footprint, a sequential chase with a 64-byte stride took 4.4 ns per access, while the random chase took 110 ns — 25 times longer. The data and the number of loads are identical; the difference is that the hardware prefetchers detect the sequential stream and bring lines in before they are requested. The benefit shrinks as the stride grows: 4.8 ns at 256 bytes but 9.4 ns at 1024 bytes, because fewer accesses fall within a page and the prefetchers stop at page boundaries and can only look a limited distance ahead.

Access granularity determines how much of the moved data is useful. The memory system always transfers whole 64-byte lines, so a program that reads 8 bytes per line gets only one eighth of what the hardware moves: with one thread, a 64-byte stride achieved 14.9 GB/s of line traffic but only 1.86 GB/s of useful data. Line bandwidth itself falls as the stride grows toward a page (from 14.9 to 4.0 GB/s), as prefetching stops helping and every access starts a new DRAM row and TLB entry.

Random access improves with block size because each DRAM access is amortized over more bytes: with six threads, random 4 KB blocks reached 25.5 GB/s, three quarters of sequential bandwidth, while random 8-byte reads managed 1.23 GB/s. Throughout, one thread reached at most 23 GB/s while six threads reached 34 GB/s; a single core cannot keep enough requests in flight to saturate DRAM, which Section 8 quantifies. One exception to the trend — random 128-byte accesses being slower than 64-byte ones — is an artifact of the benchmark code, discussed in Section 13.

7. Read/write mix

6 threads, DRAM-sized buffers. Estimated DRAM traffic adds the read-for-ownership (RFO) read of every line written with normal stores.

Mix (read %)	Seq, normal stores (GB/s)	Est. DRAM traffic	Seq, non-temporal (GB/s)	Random (GB/s)
100	34.6 ± 1.5	34.6	—	16.5
70	22.5 ± 0.3	29.2	21.8	13.0
50	18.7 ± 0.5	28.0	19.6	11.8

Author: Brendan Bianchi

Mix (read %)	Seq, normal stores (GB/s)	Est. DRAM traffic	Seq, non-temporal (GB/s)	Random (GB/s)
0 (all writes)	13.5 ± 0.5	26.9	21.1	9.95

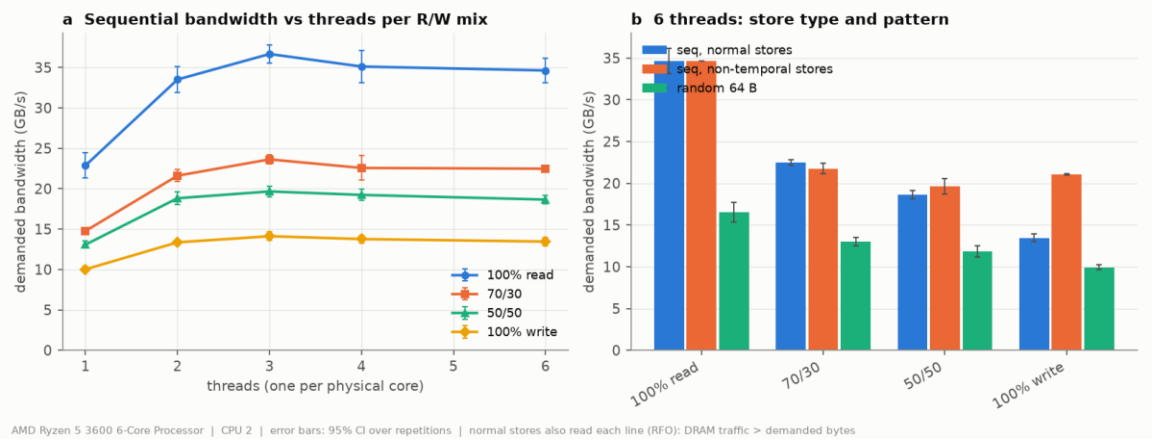


Figure 4: Demanded bandwidth and estimated DRAM traffic for each read/write mix.

With six threads streaming through DRAM-sized buffers, demanded bandwidth fell steadily as the share of writes grew: 34.6 GB/s for pure reads, 22.5 at 70/30, 18.7 at 50/50 and 13.5 GB/s for pure writes. The main reason is read-for-ownership. Before a core can write part of a line, it must own the whole line, so a normal store first reads the line from DRAM and later writes it back when it is evicted. Every byte written therefore costs about two bytes of DRAM traffic.

Accounting for that hidden read shows the memory bus was about equally busy in every case: estimated DRAM traffic stays between 27 and 35 GB/s across all mixes. Writes are not slower because DRAM writes are slow, but because the program pays for traffic it never asked for. The remaining gap below pure reads comes from the DRAM bus having to switch direction between reads and writes, which wastes cycles on every turnaround.

Non-temporal stores bypass the cache and skip the ownership read. For pure writes they raised bandwidth from 13.5 to 21.1 GB/s, a 57% improvement. At 70/30 they made no difference (21.8 vs 22.5 GB/s), because the read traffic dominates. Random access showed the same ordering at about half the bandwidth. These results support H5.

8. Intensity sweep, knee and Little's Law

```
L = \lambda W \quad\Rightarrow\quad \text{lines in flight} =
\frac{\text{bandwidth}}{64\,\text{B}} \times \text{latency}
```

8.1 Single core: independent chains (1 GiB, 2 MB pages)

Chains	Latency (ns)	GB/s	Latency inflation
1	91.7	0.70	1.00
4	100.1	2.56	1.09

Author: Brendan Bianchi

Chains	Latency (ns)	GB/s	Latency inflation
8	104.4	4.90	1.14
16	118.4 ± 11.6	8.69	1.29
24	133.5	11.51	1.46
28	151.9	11.80 (peak)	1.66
32	191.7	10.68	2.09

Little's Law at peak: 11.80 GB/s ÷ 64 B × 91.7 ns = **16.9 lines in flight** per core.

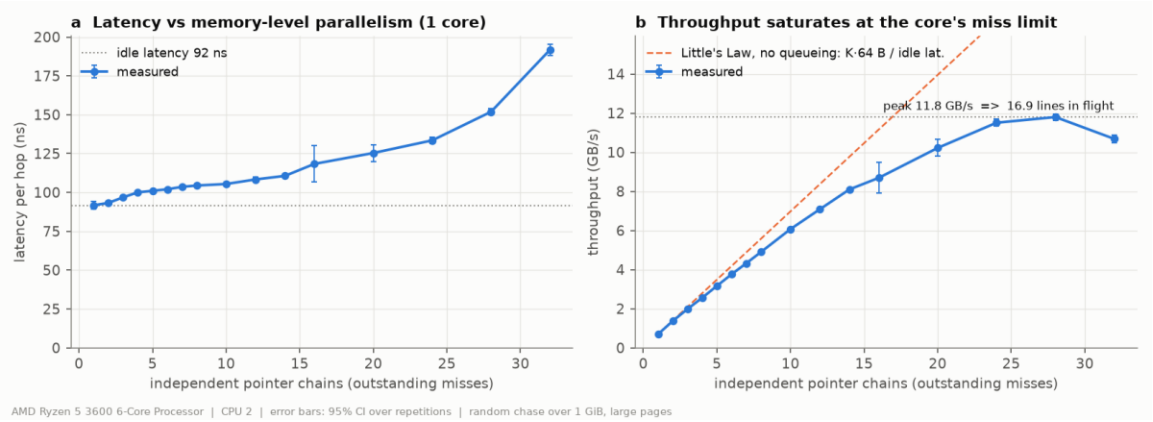


Figure 5: Single-core throughput and latency vs number of independent chains.

8.2 System: loaded latency (5 generator cores, delay swept)

Read %	Knee GB/s	Knee latency (ns)	Max GB/s	Latency at max (ns)	Lines in flight at max
100	34.8	122.5	36.0	157.5	88.6
70	19.5	153.8	22.8	237.4	84.5
50	11.4*	138.5	19.2	259.4	77.9
0	13.2	224.1	14.1	282.7	62.1

*Automatic detection, which places the knee too early. Reading Figure 6, the 50/50 curve bends at about 17 GB/s and 195 ns (see Section 13). Idle latency: 90.7 ns.

Author: Brendan Bianchi

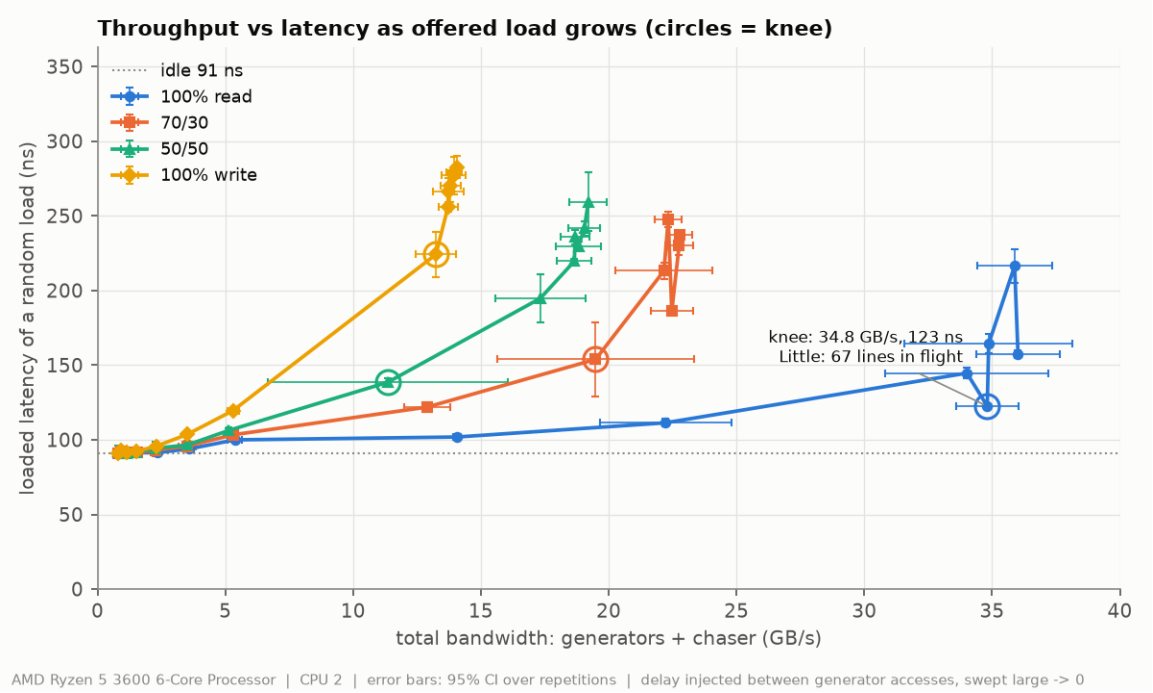


Figure 6: Loaded latency vs total bandwidth as offered load grows, per read/write mix; circles mark the automatically detected knee.

On a single core, adding independent chains raised throughput almost in proportion at first: from 0.70 GB/s with one chain to 4.90 GB/s with eight, while latency per access grew only 14%. The core was overlapping misses. Past about 16 chains, latency began to rise faster than throughput, and throughput peaked at 11.80 GB/s with 28 chains. At 32 chains throughput fell to 10.68 GB/s while latency doubled: extra requests only waited in queues. Little's Law turns the peak into a concurrency figure: 11.80 GB/s ÷ 64 B × 91.7 ns ≈ 17 lines in flight. That is the number of outstanding misses one core can track in its miss buffers and L2 request queue. It explains why a single core cannot saturate DRAM: with 17 misses each lasting about 92 ns, a core cannot exceed roughly 12 GB/s of random traffic, however fast the memory is.

The loaded-latency experiment shows the same effect at the system level. With the five other cores streaming reads, the chase latency stayed close to its idle value of 90.7 ns at light load and rose to 122.5 ns at the knee, 34.8 GB/s. Beyond it, offered load produced almost no extra bandwidth (36.0 GB/s maximum) while latency climbed to 157.5 ns. Requests were queuing at the memory controller. Applying Little's Law at the maximum gives about 89 lines in flight across the system, five times one core's limit, which is why several cores are needed to fill the memory pipeline.

Writes move the knee to lower bandwidth and higher latency: at 70% reads the knee was at 19.5 GB/s and 154 ns, and with pure writes at 13.2 GB/s and 224 ns. Read-for-ownership traffic and bus turnarounds consume capacity that does not show up as demanded bandwidth. Relative to H6, the per-core limit of about 17 misses was slightly above my 10–16 estimate. For pure reads the knee came at 34.8 GB/s, 82% of the 42.7 GB/s theoretical peak and 97% of the measured maximum, slightly later than the predicted 60–80%.

Author: Brendan Bianchi

9. Working-set sweep

The latency side of this sweep is Figure 1 (Section 5). Bandwidth is sequential with 2 MB pages; working sets are per thread. Values are the range across each region's plateau, from `data/processed/bw_wss_summary.csv`.

Region	Working set per thread	Read, 1 thread (GB/s)	Write, 1 thread (GB/s)	Read, 6 threads (GB/s)	Latency (ns, Fig. 1)
L1	8–32 KiB	80–100	25.7–27.0	462–566	1.13
L2	48–384 KiB	101–109	27.5–27.9	568–596	3.31
L3	0.75–8 MiB (1 thread); 0.75–4 MiB (6 threads)	77.4–78.8	27.2–27.7	421–443	11.2–13.2
DRAM	≥ 24 MiB (1 thread); ≥ 12 MiB (6 threads)	22.9–23.5	9.3–9.5	33.0–34.3	91–110

Transitions: 1-thread read drops from 72.9 GB/s at 12 MiB to 41.3 at 16 MiB and 23.0 at 24 MiB. The 6-thread curve drops from 430 GB/s at 4 MiB per thread to 254 at 6 MiB and 44 at 8 MiB.

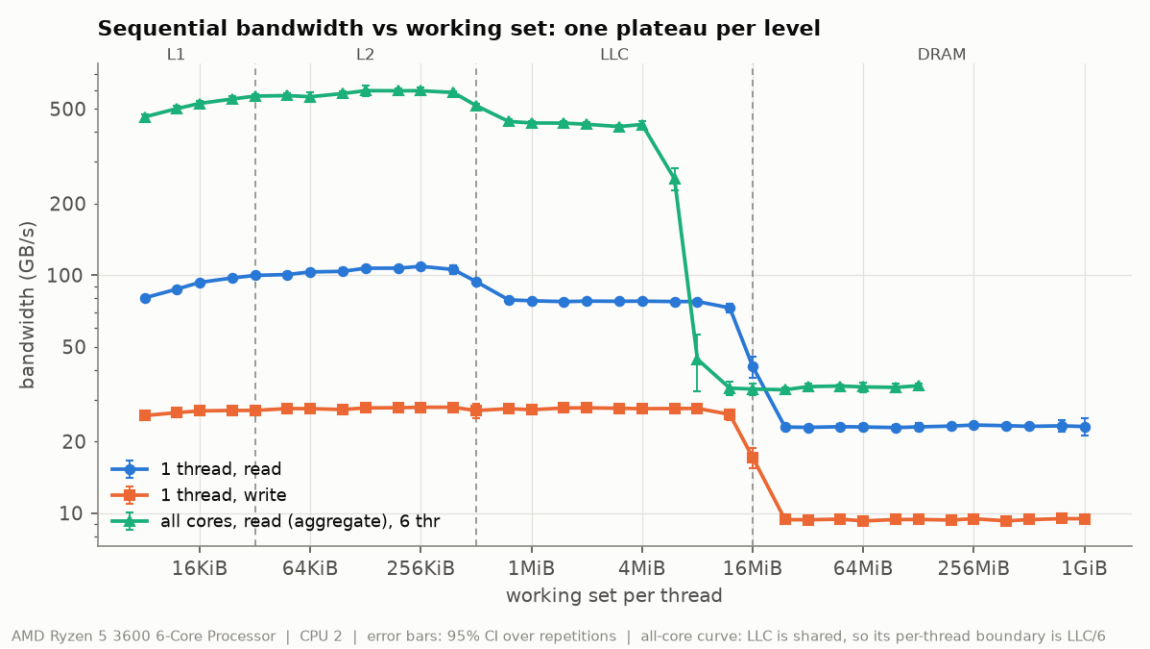


Figure 7: Sequential bandwidth vs working set per thread: 1-thread read, 1-thread write, and 6-thread aggregate read.

Sweeping the working set from 8 KiB to 1 GiB divides both the latency curve (Figure 1) and the bandwidth curve (Figure 7) into regions, but the bandwidth curve shows three clear plateaus rather than four. A single thread reads at 101–109 GB/s while its data fits in L2, about 78 GB/s from L3, and 23 GB/s from DRAM. The L1 region is not faster than L2: at 8–32 KiB the thread reaches only 80–100 GB/s. At 109 GB/s the read loop already moves about 31 bytes per cycle, so the loop itself, not the cache, is the

limit, and the very smallest buffers also pay a loop-restart overhead on every pass. Writes show the same effect even more strongly: they stay flat at about 27 GB/s, roughly one 8-byte store per cycle, from L1 all the way through L3, and fall to 9.4 GB/s only in DRAM.

The transitions are gradual rather than sharp. A working set slightly larger than a cache still finds part of its data there, replacement is not perfectly ordered, and other data such as page tables also occupies the caches. For one thread, the L3 plateau holds to about 12 MiB and collapses between 16 and 24 MiB, consistent with the 16 MiB L3 of one core complex (CCX). With six threads, the drop comes at 6 MiB per thread instead. The six threads are spread over both CCXs, three per CCX, so each CCX's 16 MiB L3 is shared three ways: about 5.3 MiB per thread. The aggregate L1/L2 bandwidth of 460–600 GB/s also shows the private caches scaling almost perfectly with core count, while DRAM does not.

At DRAM sizes, all cores together reached 34.3 GB/s, 80% of the 42.7 GB/s theoretical peak of my memory, which runs at 2667 MT/s. H7 predicted 70–85% of peak, which held. The prediction of a roughly 2× drop at each level held only loosely: L2 to L3 was 1.4× and L3 to DRAM 3.4× for one thread.

10. Cache-miss impact on kernels

768 MiB footprint, identical work per run, locality varied by visiting order. Counters are per element, from uProf.

Kernel	Order	ns/elem	GB/s	L1D miss/elem	LLC miss/elem	dTLB miss/elem
mul (z=x*y)	stride 1	1.27 ± 0.04	18.9	0.38	0.16	0.002
mul	stride 2	2.51	9.6	0.77	0.05	0.003
mul	stride 4	5.15	4.7	1.48	0.65	0.007
mul	stride 8	10.52	2.3	2.99	1.69	0.015
mul	stride 16	17.50	1.4	3.02	2.36	0.028
mul	stride 64	17.50	1.4	2.98	2.92	0.113
mul	stride 512	18.50	1.3	3.11	2.94	1.10
mul	random	26.26	1.1	3.18	2.94	2.99
saxpy (y=a*x+y)	stride 1	1.02 ± 0.02	23.5	0.26	0.09	0.001
saxpy	stride 2	2.25	10.7	0.54	0.03	0.002
saxpy	stride 4	4.07	5.9	1.03	0.33	0.004
saxpy	stride 8	7.97	3.0	2.04	1.32	0.009
saxpy	stride 16	12.44	1.9	2.05	1.93	0.018

Author: Brendan Bianchi

Kernel	Order	ns/elem	GB/s	L1D miss/elem	LLC miss/elem	dTLB miss/elem
saxpy	stride 64	11.33	2.1	1.99	1.97	0.079
saxpy	stride 512	17.67	1.4	10.47	1.99	0.76
saxpy	random	17.53	1.6	2.14	1.98	2.01

Regression of runtime on miss counts (all kernel runs):

Kernel	Runs	R^2	ns per L1D miss	ns per LLC miss	ns per dTLB miss
mul	225	0.986	1.22	4.87	2.38
saxpy	230	0.986	0.49	6.02	1.94

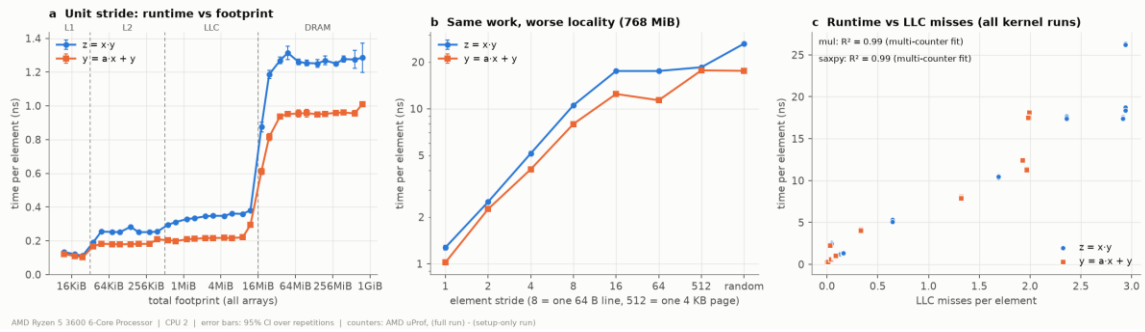


Figure 8: (a) Unit-stride runtime vs footprint; (b) runtime vs stride at 768 MiB; (c) runtime vs LLC misses per element.

To isolate the cost of misses, both kernels processed the same elements of a 768 MiB footprint with the same arithmetic. Only the order of the visits changed: unit stride, strides from 2 to 512 elements, or random. Any change in runtime is therefore caused by locality.

The counters follow the expected pattern. At unit stride, `mul` incurs 0.38 L1 misses per element, because each 64-byte line holds eight doubles and is shared by eight consecutive elements. Doubling the stride roughly doubles the misses until stride 8, where every access touches a new line. Beyond that, misses level off at about one per array per element: about 3 for `mul` (x, y and z) and 2 for saxpy (x and y). Runtime rises with them: `mul` slows from 1.27 ns per element at unit stride to 10.5 ns at stride 8, 18.5 ns at stride 512 and 26.3 ns in random order, 20 times slower for identical work.

Runtime keeps growing past stride 16 even though L1 and LLC misses are flat, for two reasons. The prefetchers stop recognizing the pattern once accesses jump across pages, so misses that were partly hidden become fully exposed. TLB misses also appear: about 1.1 per element at stride 512 and 3 per element in random order.

A least-squares fit of runtime against the three miss counts explains 98.6% of the variation for both kernels. The coefficients act as effective miss penalties in an AMAT model: about 5–6 ns per LLC miss, 0.5–1.2 ns per L1 miss and about 2 ns per TLB miss. The LLC penalty is far below the 91 ns idle DRAM latency because a streaming kernel overlaps many misses (Section 8) and prefetching hides part of each

one. A few counter values do not fit this picture neatly; they are listed in Section 13. Overall, the results support H8.

11. TLB-miss impact and huge pages

11.1 One line per 4 KB page, random page order

Pages touched	Address span	Lines footprint	4 KB pages (ns)	2 MB pages (ns)	dTLB miss/access, 4 KB	dTLB miss/access, 2 MB
16	64 KiB	1 KiB	1.15	1.15	0.00	0.00
256	1 MiB	16 KiB	1.58	1.16	0.00	0.00
1,024	4 MiB	64 KiB	5.17	3.50	0.00	0.00
4,096	16 MiB	256 KiB	5.67	3.61	0.01	0.00
8,192	32 MiB	512 KiB	13.80	10.68	0.09	0.00
16,384	64 MiB	1 MiB	19.87	11.14	0.61	0.00
65,536	256 MiB	4 MiB	28.40	12.48	0.93	0.00
131,072	512 MiB	8 MiB	98.03	72.07	0.97	0.00
262,144	1 GiB	16 MiB	115.29	92.56	1.03	0.00

At 262,144 pages, 4 KB pages cost 22.7 ns more per access (≈ 22 ns per page walk). At 16 pages the difference is 0.

11.2 Kernels: 4 KB vs 2 MB pages (ns/elem)

Kernel	Order	4 KB	2 MB	Speedup
mul	stride 1	1.232	1.203	1.02×
mul	stride 512	18.35	17.55	1.05×
mul	random	26.20	22.27	1.18×
saxpy	stride 1	1.045	0.987	1.06×
saxpy	stride 512	17.50	19.27	0.91×
saxpy	random	17.58	16.05	1.10×

Author: Brendan Bianchi

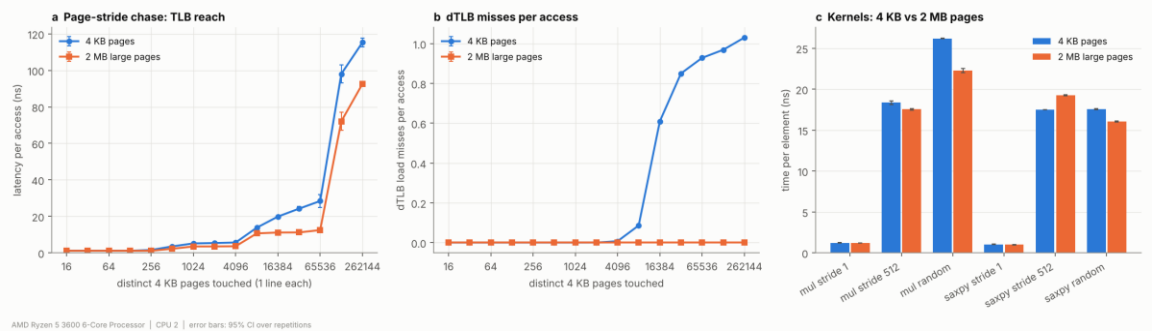


Figure 9: (a) Latency vs pages touched, 4 KB vs 2 MB pages; (b) dTLB misses per access; (c) kernels with 4 KB vs 2 MB pages.

To separate TLB effects from cache effects, this experiment touches exactly one cache line in each 4 KB page, visiting the pages in random order. As the number of pages grows, the address range the TLB must cover grows 64 times faster than the data the caches must hold: at 4,096 pages the span is 16 MiB but the lines occupy only 256 KiB. Any latency added in that range comes from address translation.

The 4 KB curve steps up as the page count passes the reach of each TLB level. Zen 2's L1 DTLB has 64 entries and its L2 DTLB 2,048 entries (7-cpu.com), covering 256 KiB and 8 MiB with plain 4 KB pages. Latency rises from 1.15 to 5.2 ns between 256 and 1,024 pages, as accesses start missing the L1 DTLB and are served by the slower L2 DTLB. Counted page walks stay near zero up to 4,096 pages and begin around 8,192 pages (32 MiB), well beyond the L2 DTLB's nominal 8 MiB reach. Zen 2 is reported to combine up to four adjacent 4 KB translations into one entry, which would extend the reach to 32 MiB and matches where the walks begin. From 16,384 pages on, nearly every access needs a page walk (0.61 rising to 1.03 per access). With 2 MB pages, page walks stay at zero across the whole sweep; the latency growth that remains appears only once the line footprint outgrows each cache, so it is a cache effect rather than translation.

At 262,144 pages (1 GiB), 4 KB pages cost 22.7 ns more per access than 2 MB pages. With about one walk per access, that is roughly 22 ns per page walk, consistent with the 19 ns difference found for the plain DRAM chase in Section 5. Inside TLB reach (16 pages) the two page sizes are identical, confirming H2.

For real kernels the benefit depends on the access pattern. Huge pages helped most in random order (18% for `mul`, 10% for saxpy), where every access can need a new translation. They barely helped at unit stride (2–6%), because a sequential sweep reuses each 4 KB translation for 512 consecutive elements. At stride 512, saxpy even ran 9% slower with huge pages. Huge pages are not a universal speed-up, which makes this one of the project's negative results (Section 12). The sudden jump at 131,072 pages for both page sizes is discussed in Section 13.

12. Benchmark pitfalls and failure cases

Each pitfall runs the same code measured the naive way and the correct way.

Pitfall	Naive measurement	Correct measurement	Effect shown?

Author: Brendan Bianchi

Pitfall	Naive measurement	Correct measurement	Effect shown?
P1 LLC-resident buffer reported as "DRAM bandwidth"	421.9 GB/s	34.4 GB/s (DRAM-sized)	Yes, 12× overstatement
P2 Reading untouched memory	32.3 GB/s (first rep 28.4)	33.4 GB/s (initialized)	Weak on Windows
P2 First-touch write faults	9.64 GB/s (first rep 8.47)	9.89 GB/s (prefaulted)	First repetition only
P3 Sequential chase reported as "DRAM latency"	4.38 ns	111.1 ns (random)	Yes, 25× understatement
P4 Unpinned, oversubscribed loaded latency	555 ns (CV 11.6%)	217 ns pinned (CV 2.9%)	Yes, 2.6× and noisy
P4 Unpinned bandwidth	33.7 GB/s	33.9 GB/s pinned	No
P4 Generator on SMT sibling vs other core	106.5 ns	108.2 ns	No
P5 Nominal GHz instead of calibrated	4.07 cycles	4.02 cycles	No (boost off)

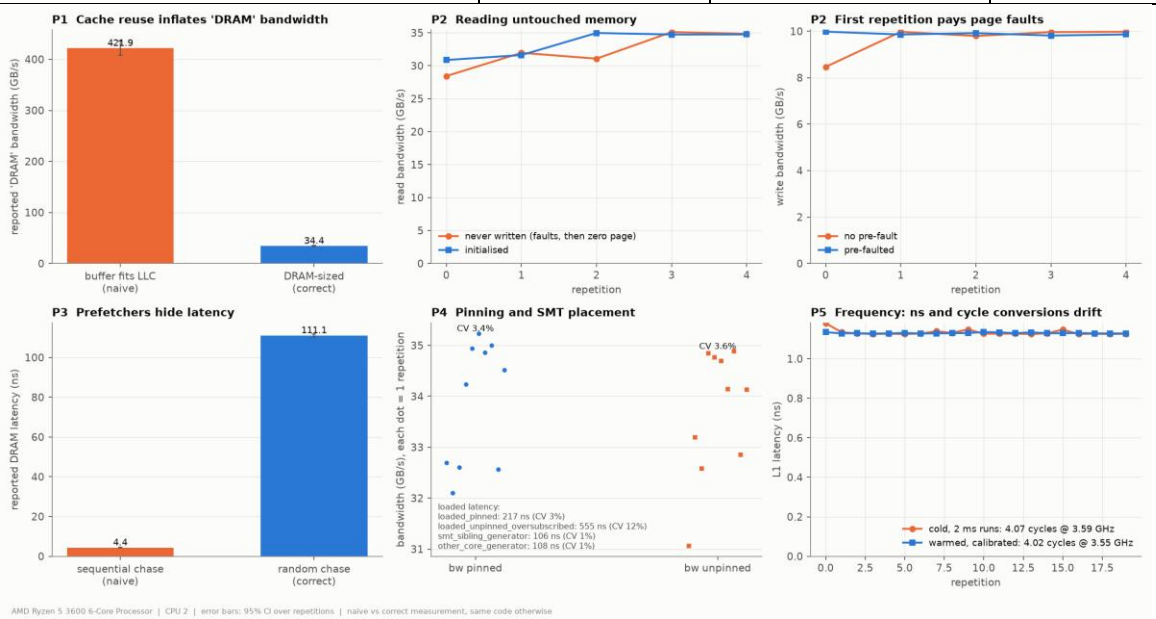


Figure 10: Naive vs correct measurement for pitfalls P1–P5.

P1 – Cache-resident "DRAM" bandwidth. A buffer that fits in the caches reported 421.9 GB/s, more than ten times what DRAM can deliver; the correctly sized buffer measured 34.4 GB/s. The data never left the caches, so the test measured L2/L3 bandwidth under a DRAM label. The suite avoids this by sizing every DRAM test well beyond the 16 MiB L3.

P2 – Untouched memory and first-touch faults. Reading never-written memory gave 32.3 GB/s versus 33.4 GB/s for initialized memory, and writing to fresh pages 9.64 versus 9.89 GB/s after prefaulting. In both cases the penalty appears only in the first repetition (28.4 and 8.47 GB/s), because Windows allocates and zeroes each page the first time it is touched. The effect is small here because the suite runs warm-up repetitions; a one-shot benchmark would have reported the slower number. The suite initializes buffers from the owning thread before timing.

P3 – Sequential chase as "memory latency". A sequential pointer chase reports 4.38 ns, while the random chase over the same memory reports 111.1 ns. The sequential version measures the prefetchers, not DRAM, and understates latency 25-fold. The suite uses a random single-cycle permutation for all latency numbers.

P4 – Thread placement. Letting the scheduler place an oversubscribed set of generator threads raised the chase's loaded latency from 217 to 555 ns, with a coefficient of variation of 11.6% instead of 2.9%. The measurement thread competed for its own core and its results became both wrong and noisy. All other tests pin their threads.

P5 – Nominal vs measured clock. Converting nanoseconds to cycles with the nominal clock gave 4.07 cycles for L1 versus 4.02 with the calibrated clock. The error is small only because boost was disabled; with boost enabled the core runs well above its nominal frequency and the conversion would be visibly wrong. The suite measures the clock in every run.

Negative results and failure cases

The clearest failure case is huge pages. Section 11 showed they remove page walks entirely, yet for working sets within TLB reach they changed nothing (1.15 ns either way at 16 pages), and for a unit-stride kernel they improved runtime by only 2–6%. A sequential sweep already reuses each translation hundreds of times, so there is little translation cost to remove, and at stride 512 saxpy was even 9% slower with huge pages. Their benefit is limited to large footprints accessed irregularly.

A second failure case is memory-level parallelism beyond the core's limit. Adding chains raised single-core throughput only up to about 28; at 32 chains throughput dropped from 11.80 to 10.68 GB/s while latency doubled. Past the limit of about 17 outstanding misses, more parallelism in the code only creates queueing.

Finally, two safeguards made their own pitfalls disappear. Pinning made no measurable difference to streaming bandwidth (33.9 vs 33.7 GB/s), and a generator on the measurement core's SMT sibling hurt the chase no more than one on another core (106.5 vs 108.2 ns), because the chase has only one miss in flight and doesn't compete for the core's miss buffers. These are honest negative results: the pitfalls exist, but this workload and fixed clock did not trigger them.

13. Anomalies, limitations and threats to validity

13.1 Anomalies

#	Observation	Where	Likely explanation (state as hypothesis)	Possible follow-up test

Author: Brendan Bianchi

#	Observation	Where	Likely explanation (state as hypothesis)	Possible follow-up test
A1	Random 128 B reads slower than 64 B (3.2 vs 4.4 GB/s)	Sec. 6	The 64 B path uses a short vector loop; other sizes use a longer scalar loop, so fewer misses fit in the out-of-order window at once	Vectorize the 128 B path and re-run
A2	Automatic knee for the 50/50 mix (11.4 GB/s) sits well before the visible bend (about 17 GB/s, 195 ns)	Sec. 8.2	Coarse delay grid: offered load jumps from about 4.5 to 11 to 17 GB/s between three delay steps, and the detector picks the first point past a large gap	Add delay points between 80 and 300
A3	Large CI at 16 chains (±11.6 ns)	Sec. 8.1	Confirmed outlier: repetition 4 measured 134.7 ns vs 112.7–117.9 ns for the other four (likely an interrupt or background task)	Report the median, or re-run that point
A4	LLC misses lower at stride 2 than stride 1 (0.05 vs 0.16)	Sec. 10	Demand-miss counter excludes prefetched lines; counts reflect prefetch timeliness, not traffic	Count prefetch fills too
A5	saxpy stride 512: 10.5 L1D misses/elem (expected about 2) and slower with 2 MB pages (0.91×)	Sec. 10, 11	Counter counts retried/repeated misses; possible conflict effect when x and y map to the same sets	Pad arrays and re-run
A6	2 MB pages: latency jumps 12.5 → 72 ns at an 8 MiB line footprint, below the 16 MiB L3	Sec. 11	Each access pulls an unused neighbor line; L3 holds L2 evictions, so the effective footprint doubles to about 16 MiB	Disable adjacent-line prefetch, or test 6 and 12 MiB points
A7	Few dTLB misses per element at stride 512 for mul (1.1, expected about 3)	Sec. 10	Page-table entries cached and coalesced; L2 TLB covers part of the walk	Compare with random order (2.99)
A8	No separate L1 bandwidth plateau: 8–32 KiB reads (80–100 GB/s) are slower than L2 (101–109 GB/s); writes flat at 27 GB/s through L3	Sec. 9	The benchmark loop, not the cache, is the limit (about 31 B/cycle for reads, about one 8-byte store per cycle for writes), plus loop-restart overhead on tiny buffers	Use wider (AVX) loads/stores and unrolling

Author: Brendan Bianchi

#	Observation	Where	Likely explanation (state as hypothesis)	Possible follow-up test
A9	Memory runs at 2667 MT/s although the DIMMs are rated 3200 MT/s	Sec. 3.1	XMP profile not enabled in the BIOS; all DRAM numbers are for 2667 MT/s	Optional: enable XMP and repeat exp4/exp5

A1. I expected random-access bandwidth to rise steadily with block size, but 128-byte blocks (3.22 GB/s) were slower than 64-byte ones (4.40 GB/s). The benchmark handles 64-byte accesses with a short vectorized loop and all other sizes with a longer scalar loop. More instructions per access means fewer misses fit in the core's out-of-order window at once, so fewer requests overlap. This is an artifact of the benchmark code rather than the hardware, and I am fairly confident of it because the 256-byte and larger sizes, which use the same scalar loop, follow the expected trend among themselves.

A2. For the 50/50 mix, the automatic knee detector placed the knee at 11.4 GB/s and 138 ns, but Figure 6 shows the curve bending later, at about 17 GB/s and 195 ns. The delay sweep is coarse in that range: going from a delay of 300 to 150 to 80 raises the offered load from about 4.5 to 11 to 17 GB/s in single steps. With so few points the detector picks the first point past the large jump. I report the visual knee for that mix; adding delay values between 80 and 300 would resolve it.

A3. The 16-chain point has a confidence interval of ±11.6 ns, ten times wider than its neighbors. The raw rows show why: four repetitions measured 112.7–117.9 ns, but the fifth measured 134.7 ns. A single disturbed repetition, most likely an interrupt or background task, inflated both the mean and the interval; the median (113.6 ns) is the better estimate for that point.

A4. `mul` showed fewer LLC misses at stride 2 (0.05 per element) than at stride 1 (0.16). The counter records demand misses only; at small strides the prefetchers fetch most lines before the program asks for them, so the count reflects how early the prefetches arrive rather than the actual DRAM traffic. I treat small-stride LLC counts as low-confidence.

A5. At stride 512, saxpy recorded 10.5 L1 misses per element where about 2 were expected, and it ran 9% slower with huge pages. Both x and y are then accessed at the same offset within their pages, which may cause repeated conflicts in the same cache sets, and the counter may also count retried misses. Padding one array and re-running would test this; my confidence in the explanation is low.

A6. With 2 MB pages, latency jumped from 12.5 to 72 ns once the line footprint reached 8 MiB, although the L3 holds 16 MiB. I checked that the lines are spread evenly over the cache sets, so set conflicts are not the cause. My hypothesis is that each access also brings in a neighboring line that is never used. Because Zen 2's L3 is filled with lines evicted from L2, those unused neighbors occupy L3 space too, doubling the effective footprint to about 16 MiB. Measuring points between 4 and 8 MiB, or disabling the adjacent-line prefetcher, would test it.

A7. At stride 512, `mul` recorded about 1.1 page walks per element where three arrays suggested three. Walk results and page-table entries are themselves cached, and adjacent translations can be combined, so some accesses avoid a full walk. In random order, where no such reuse exists, the count is 2.99 as expected.

A8. I expected the bandwidth sweep to show its highest plateau in L1, but single-thread reads at 8–32 KiB (80–100 GB/s) were slower than in L2 (101–109 GB/s), and writes stayed at about 27 GB/s from L1 through L3. At these speeds the benchmark loop is the bottleneck: 109 GB/s is about 31 bytes per cycle, well under the two 32-byte loads per cycle Zen 2 can issue, and 27 GB/s is about one 8-byte store per cycle. For the smallest buffers the loop also restarts often, adding overhead. The L1 bandwidth is therefore higher than this benchmark can show; wider vector loads and stores would be needed to measure it. I am fairly confident in this explanation because the latency test, which does not depend on loop throughput, shows L1 clearly.

A9. The system report shows the memory modules are rated for 3200 MT/s but run at 2667 MT/s, because the BIOS memory profile (XMP) is not enabled. This is not an error in the measurements, but it means every DRAM bandwidth and latency figure here applies to 2667 MT/s, and the theoretical peak is 42.7 GB/s rather than 51.2 GB/s.

13.2 Limitations and threats to validity

Several limitations bound how far these results generalize. Counter values come from sampling, so small differences between configurations carry noise that the runtime confidence intervals do not capture. All measurements were taken on one machine with one memory configuration and with boost disabled; a boosted CPU, faster memory or a multi-socket server would give different numbers, although the methods would carry over. Windows background activity cannot be removed entirely, and its effect shows up only as wider confidence intervals on some points. The L3 experiments stayed within one core complex, so traffic between the two complexes was not studied. Finally, the Windows "Memory integrity" security feature had to be disabled for profiling, so the measured system was not in its default configuration.

14. Conclusions

On the Ryzen 5 3600, each step down the memory hierarchy costs roughly 3–8 times more latency than the one above, ending at about 320 cycles for a DRAM access, plus about 20 ns more when 4 KB pages force a page walk. A single core can keep only about 17 misses in flight. By Little's Law that caps it near 12 GB/s for random access, so single-threaded code is limited by latency and concurrency rather than by DRAM bandwidth. With all cores, reads reach about 35 GB/s, where queueing at the memory controller takes over: latency stays near idle until close to that limit and then rises steeply. Writes cost about twice their size in traffic unless non-temporal stores avoid the ownership read. For software, locality is decisive: the same kernel ran 10–20 times slower when only its access order changed, and three miss counters explained 98.6% of that variation. A natural next step would be to repeat the study with boost enabled and across both core complexes.

Appendix A: Reproducing the results

Run in the "MSYS2 UCRT64" terminal opened as administrator:

1. `./scripts/setup_env.sh` — boost off, processor state 100%, "Lock pages in memory" granted (sign out and back in once).
2. Turn off Memory integrity (needed for uProf sampling) and reboot.
3. `./scripts/uprof_probe.sh` — confirms counters work.

4. `QUICK=1 ./scripts/run_all.sh` — smoke test, about 20–30 minutes.
5. `./scripts/run_all.sh` — full run, about 1.5–2 hours; outputs in `data/raw/`, `data/processed/summary.md` and `figures/`.
6. `./scripts/restore_env.sh`, then turn Memory integrity back on.

The first full run stopped during the loaded-latency experiment: five generator threads each requested 256 MiB of 2 MB pages, and Windows reported error 1450 because not enough contiguous physical memory was free. Generators only produce traffic, so they now use 4 KB pages, and bandwidth buffers fall back to 4 KB pages with a warning if large pages run out. A configuration that still fails is logged to `data/raw/failures.txt` and skipped instead of stopping the run. After a reboot, the remaining experiments completed with no failures.

Appendix B: Repository layout

Path	Contents
`src/`	C++ benchmarks: `latency`, `bandwidth`, `loaded`, `kernels`, `sysinfo`
`scripts/`	Environment setup/restore, validation, experiments `exp1`–`exp8`, `run_all.sh`
`analysis/`	`process.py` (tables, checks, summary), `plot.py` (figures), `uprof_merge.py` (counters)
`data/raw/`	Raw per-repetition CSVs, system info, uProf sessions
`data/processed/`	Summary tables and `summary.md`
`figures/`	fig1–fig10 PNGs
`bin/BUILD_INFO`	Compiler version and flags of the measured build

Appendix C: References

- AMD, *Software Optimization Guide for AMD Family 17h Models 30h and Greater Processors* (Zen 2 microarchitecture).
- AMD, *AMD uProf User Guide* (event-based sampling and event definitions).
- 7-cpu.com, AMD Zen 2 cache, TLB and memory latency figures.
- Little, J. D. C., "A Proof for the Queuing Formula L = λW," *Operations Research*, 9(3), 1961.